Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

Are Large Language Models Ready for Quantum Software Engineering? A Multivocal Literature Review

Antonio Della Porta

adellaporta@unisa.it

University of Salerno https://orcid.org/0000-0003-1860-8404

Stefano Lambiase

stla@cs.aau.dk

Aalborg University https://orcid.org/0000-0002-9933-6203

Andrea De Lucia

University of Salerno https://orcid.org/0000-0002-4238-1425

Fabio Palomba

University of Salerno https://orcid.org/0000-0001-9337-5116

Systematic Review

Keywords: Software Engineering, Quantum, Literature Review, Large Language Models

Posted Date: June 24th, 2026

DOI: https://doi.org/10.21203/rs.3.rs-10126259/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: The authors declare no competing interests.

Are Large Language Models Ready for Quantum Software Engineering? A Multivocal Literature Review

Antonio Della Porta · Stefano Lambiase · Andrea De Lucia · Fabio Palomba

Received: date / Accepted: date

Abstract Quantum computing is steadily transitioning from theoretical exploration to experimental deployment, bringing with it distinct engineering challenges. This shift has led to the emergence of Quantum Software Engineering (QSE), a research area that adapts and extends software engineering principles to the quantum domain. In parallel, Large Language Models (LLMs) have become widely adopted in classical software development workflows. Although their potential to assist QSE tasks is increasingly discussed, existing evidence remains fragmented across research venues and practitioner sources. This study presents a Multivocal Literature Review that systematically synthesizes peer-reviewed and grey literature to characterize how LLMs are used in QSE and assess their reported limitations. We identified and analyzed 24 primary sources, mapped their contributions onto the quantum software development lifecycle, examined the distribution of adopted LLM families, and synthesized recurring limitations. The results show that LLM adoption in QSE is predominantly code-centric, with most studies focusing on program synthesis, repair, and refactoring, while lifecycle phases such as requirements engineering, architectural reasoning, and comprehensive validation receive limited attention. Reported challenges include semantic inaccuracies, execution fail-

A. Della Porta;
University of Salerno, Via Giovanni Paolo II, 132, 84084 Fisciano (SA), Italy
E-mail: adellaporta@unisa.it

S. Lambiase
Aalborg University in Copenhagen, A. C. Meyers Vænge 15, 2450 Copenhagen, Denmark
E-mail: stla@cs.aau.dk

A. De Lucia
University of Salerno, Via Giovanni Paolo II, 132, 84084 Fisciano (SA), Italy
E-mail: adelucia@unisa.it

F. Palomba
University of Salerno, Via Giovanni Paolo II, 132, 84084 Fisciano (SA), Italy
E-mail: fpalomba@unisa.it

ures on real quantum backends, sensitivity to prompt formulation, and limited domain coverage. Moreover, the empirical landscape is largely shaped by proprietary models, raising concerns regarding reproducibility and longitudinal comparability. Overall, current evidence suggests that LLMs function primarily as assistive tools for bounded tasks rather than as robust, lifecycle-spanning engineering agents in quantum software development.

Keywords Software Engineering · Quantum · Literature Review · Large Language Models

1 Introduction

Quantum computing is becoming an important technology for solving problems that classical computers cannot handle efficiently. However, the transition from theoretical advances in quantum algorithms to reliable, large-scale quantum applications requires disciplined engineering practices. This need has led to the emergence of a new field of research, Quantum Software Engineering (QSE), which aims to adapt and extend classical software engineering (SE) principles to the quantum domain (Murillo et al. 2025).

Despite the efforts of the QSE community, studies show that quantum programming remains difficult in practice. Developers face challenges related to unstable and rapidly evolving APIs, low-quality documentation, complex compilation pipelines, and difficulties in debugging and testing quantum programs (De Stefano et al. 2022; Li et al. 2021). These issues are further amplified by the scarcity of expertise and the immaturity of current tools and infrastructures.

In such a context, where complexity is high and support mechanisms are still evolving, Large Language Models (LLMs) may represent a promising form of support. Models like GPT (OpenAI), Gemini (Google Deepmind), and LLaMA (Meta Labs) have already achieved strong results in traditional SE. Recent surveys and systematic reviews report that LLMs can support a broad range of development activities, including code generation, program comprehension, documentation, testing, and maintenance tasks (Fan et al. 2023; Hou et al. 2024). Because they can process both natural and programming languages, LLMs can reduce the effort required to interact with complex APIs and tooling, making them attractive for domains with steep learning curves.

In quantum computing, the potential is appealing, but the risks are also amplified. Quantum programs are brittle with respect to API changes and backend constraints, and incorrect code can be hard to detect due to stochastic outcomes and limited debugging support (Murillo et al. 2025; Paltenghi and Pradel 2024). This has already motivated domain-specific efforts, such as training or adapting code LLMs for quantum programming (Dupuis et al. 2024), proposing quantum-focused benchmarks to evaluate code generation (Vishwakarma et al. 2024), and exploring LLM-based quality assurance support (e.g., linting for quantum-specific issues) (Shin et al. 2025). In parallel, industrial tools are emerging, such as Qiskit Code Assistant, which relies on

a quantum-adapted LLM to provide code suggestions for Qiskit users (IBM Quantum 2025).

Despite the growing interest in LLMs for QSE, the available evidence is scattered across venues and communities and targets heterogeneous QSE activities and lifecycle phases, making it difficult to understand what has been tried, what works, and what remains open. Existing QSE roadmaps and surveys provide useful high-level perspectives on QSE challenges and quality assurance. However, they do not provide a comprehensive account of (1) the specific tasks for which LLMs are employed within QSE, (2) the LLMs most commonly adopted to support QSE activities, and (3) the limitations associated with these tools and their application in the QSE domain. **Therefore, this study aims to systematically map and synthesize existing evidence on the use of LLMs in QSE, identifying the tasks they support, the models most frequently adopted, and the limitations reported in the literature.**

To address the aforementioned objective, we conducted a Multivocal Literature Review (MLR), a specific form of Systematic Literature Review (SLR) that integrates evidence from both peer-reviewed research and grey literature (Garousi et al. 2016, 2019). In rapidly evolving tool ecosystems, a substantial portion of actionable knowledge is disseminated outside traditional academic venues, including tool documentation, technical reports, blog posts, and practitioner discussions. Prior research in Software Engineering (SE) highlights that excluding such sources may result in an incomplete and potentially biased understanding of the state of practice. An MLR is therefore particularly suitable in this context, as it enables the systematic consolidation of scientific and practitioner evidence (Garousi et al. 2016, 2019). By adopting this approach, we aim to provide a comprehensive and structured overview of LLM usage in Quantum Software Engineering (QSE), reduce fragmentation and duplicated efforts, and establish a shared evidence base to support informed and responsible adoption of LLMs in quantum software development (Fan et al. 2023; Hou et al. 2024). The contributions of this review are threefold:

1. We provide a classification of QSE activities in which LLMs are applied, showing which phases of the software lifecycle receive the most attention and which remain underexplored.
2. We summarize which LLMs and adaptation strategies are used across studies, highlighting the balance between closed-source and open-source alternatives and the implications for reproducibility and reuse (Fan et al. 2023; Hou et al. 2024).
3. We analyze the limitations reported in the literature (e.g., correctness, executability, evaluation gaps, and tool- or platform-dependence), offering insights into current barriers and outlining directions for future research, including the need for benchmarks and rigorous evaluation protocols (Hou et al. 2024; Paltenghi and Pradel 2024; Vishwakarma et al. 2024).

The paper is structured as follows. In Section 2 reviews related work and provides the background for the study. Section 3 details the research design

and the MLR protocol. Section 4 presents the study results and answers the research questions. Section 5 discusses the findings, Section 6 outlines the main limitations, and Section 7 concludes the paper.

2 Background and Related Works

In the following subsections, we summarize the most relevant literature regarding QSE.

2.1 Quantum Software Engineering (QSE)

Quantum computing, a subfield of computer science, harnesses the principles of quantum mechanics to perform computations (Hoare and Milner 2005; Knight 2018). After this, QSE has emerged as a formal research discipline (Moguel et al. 2020; Piattini et al. 2020; Zhao 2020), with its foundational principles outlined in the *Talavera Manifesto* (Piattini et al. 2020). Building upon this work, Piattini et al. (2021) have further defined specific research domains within QSE, identifying four key areas: the design of quantum-hybrid systems, the development of testing methodologies, the evaluation of quantum program quality, and the re-engineering of classical-quantum information systems. A central challenge highlighted in their work is the need to bridge the knowledge gap between quantum computer scientists and traditional software engineers, as collaboration between these fields is crucial for the advancement of QSE.

The work by Piattini et al. (2021) has shaped subsequent research in the field of QSE. In the years that followed, the research community started to focus on various topics, like testing and maintenance techniques (e.g., Ali 2024; García de la Barrera et al. 2023; Leite Ramalho et al. 2025; Yu and Zhao 2025), architectural design principles (e.g., Zhao et al. 2024), and the characterization of development processes and associated challenges (e.g., Dwivedi et al. 2024; Zappin et al. 2025). More recently, increasing attention has also been devoted to security and vulnerability concerns in quantum software systems (e.g., Barletta et al. 2025; Mao et al. 2025; Nadim et al. 2025).

Expanding on the work by Piattini et al. (2021), De Stefano et al. (2024a) conducted a systematic mapping study to assess the current research landscape in QSE. Their study revealed that much of the existing research has focused disproportionately on software testing, highlighting a lack of studies on the factors influencing the adoption of QSTs—an area that remains relatively unexplored. Moreover, contributors (Unitary Fund 2024) and researchers (De Stefano et al. 2024b; Shaydulin et al. 2020) have emphasized the need for greater attention to these aspects, particularly given the interdisciplinary nature of quantum software development.

Recently, Murillo et al. (2025) proposed a research roadmap, identifying key gaps in QSE. Their study outlined several critical challenges that the QSE community should prioritize, including: ensuring quality assurance in quantum systems through improved testing and debugging methodologies (Long and Zhao

2024; Ye et al. 2023); shifting towards a service-oriented approach to quantum software development (Romero-Álvarez et al. 2023; Valencia et al. 2021); exploring model-driven quantum development (Ali and Yue 2020; Gemeinhardt et al. 2021); and deepening research on software engineering processes, particularly in the management of teams developing QST-based systems, which are inherently socio-technical in nature.

To support developers, quantum programming frameworks have been introduced. Qiskit (Qiskit contributors) is among the most widely used, offering tools for circuit design, simulation, and hardware execution, while Cirq (Google Quantum contributors) emphasizes circuit optimization and backend integration. PennyLane has become a reference point for hybrid quantum–classical programming and machine learning tasks, whereas Q# (Microsoft) provides a domain-specific language integrated into the .NET ecosystem.

2.2 Large Language Models (LLMs) in QSE

In recent years, the SE field has been significantly reshaped by the rise of Large Language Models (LLMs) (Fan et al. 2023; Hou et al. 2024). Compared with the rich body of work in classical SE (Hou et al. 2024), the use of LLMs in quantum software engineering remains at an early stage, with only a handful of pioneering studies. Some works have explored code generation, for instance, showing that a retrieval-augmented approach in PennyLang can significantly improve the correctness of LLM-produced programs (Basit et al. 2025). Others investigated program comprehension, finding that LLaMA and GPT-3.5 can generate explanations of quantum circuits but often with limited clarity or depth (d'Aloisio et al. 2024)(Shin et al. 2025). Research on refactoring and migration applied LLMs to the task of classifying the modifications needed when moving across Qiskit versions, combining expert-defined taxonomies with those suggested by the model (Suárez et al. 2025b). More recently, LintQ-LLM demonstrated that LLMs can extend traditional static analysis by flagging quantum-specific issues that conventional tools overlook (Campbell et al. 2025).

The current evidence on LLM usage in QSE is fragmented across software engineering venues, quantum computing venues, and tool-oriented outlets, which makes it difficult to derive a consolidated picture of which QSE activities are supported, how success is measured, and what limitations repeatedly occur (Murillo et al. 2025; Paltenghi and Pradel 2024; Sarkar 2024). In addition, for fast-moving technical areas, practice-facing knowledge is frequently reported outside peer-reviewed literature, for example, in tool documentation, technical reports, engineering blogs, and practitioner discussions. MLR are explicitly proposed in software engineering as a way to systematically integrate peer-reviewed studies with grey literature when the state of practice is relevant and would otherwise be missed (Garousi et al. 2016, 2019). This motivation directly applies to LLMs in QSE: the tool landscape changes quickly, industrial and open source ecosystems influence what developers actually do,

and academic studies alone may not capture emerging practices or recurring pain points early enough. An MLR can therefore provide a more complete baseline for the field by synthesizing both research and practitioner evidence on where LLMs are applied in QSE, how they are evaluated, and which limitations and risks are most consistently reported (Garousi et al. 2016, 2019).

Related Work: Summary and Research Gap.

Existing studies suggest that LLMs can support several QSE activities, but the evidence is fragmented and mostly exploratory. The field still lacks a consolidated view of how LLMs are applied in QSE and which limitations recur. This review addresses this gap by synthesizing current applications, the models used, and the main reported challenges.

3 Research Study Design

To achieve our goal, we performed a MLR (Garousi et al. 2016), a research process where past published formal literature (e.g., journal and conference papers) and grey literature (e.g., blog posts, videos, and whitepapers) on the investigated topic are systematically identified, selected, and critically assessed. An MLR builds on top of the concept of Systematic Literature Review (SLR) (Kitchenham et al. 2010), where only formal literature is considered, to provide benefits not only to the research community but also to practitioners.

We relied on two guidelines when defining the protocol and conducting the review. For the formal literature analysis, we adopted the well-established approach by Kitchenham et al. (2010). In addition, we followed the guidelines by Garousi et al. (2019) for identifying, selecting, and assessing grey literature. In the following subsections, we describe the main steps of each phase.

3.1 Planning the Review

3.1.1 Initiation Phase

The Initiation phase consisted of three sub-phases.

Initial Examination of Previous Studies. To gain initial familiarity with the topic and reduce the risk of missing relevant terminology, we conducted a preliminary examination of a small seed set of studies before running the main search. In this step, we reviewed representative secondary sources on QSE to understand the main concepts, typical tasks, and evaluation practices that could influence our review protocol. The seed studies were (Murillo et al. 2025) and (Paltenghi and Pradel 2024), two recent surveys that structure QSE around lifecycle activities such as testing, analysis, and quality assurance. From these works, we derived the core terminology used

Table 1: Checklist to decide whether to conduct an MLR proposed by Garousi et al. (2019).

#	Question	Answer
1	Is the subject "complex" and not solvable by considering only the formal literature?	Yes
2	Is there a lack of volume or quality of evidence, or a lack of consensus of outcome measurement in the formal literature?	Yes
3	Is the contextual information important to the subject under study?	Yes
4	Is the goal to validate or corroborate scientific outcomes with practical experiences?	Yes
5	Is the goal to challenge assumptions or falsify results from practice using academic research or vice versa?	Yes
6	Would a synthesis of insights and evidence from industrial and academic community be useful to one or both communities?	Yes
7	Is there large volume of practitioner sources interest, indicating high practitioners interest in the topic?	Yes

to describe the domain, particularly the explicit framing of "quantum" in combination with "software engineering" and "software development", which informed the first two blocks of the search string. Building on their discussion of tooling and automation challenges, we then intersected this vocabulary with terms related to LLMs and Generative AI to systematically capture studies at the intersection of the two areas.
Following MLR and SLR guidance, we treated this activity as a throw-away prototyping step: insights were used only to refine the search strategy and clarify eligibility criteria, while the examined sources were not included as part of the main selection pool to avoid biasing the systematic process (Garousi et al. 2019; Kitchenham et al. 2010).

Motivation and Needs Identification. Before deciding to conduct an MLR, Garousi *et al.* recommend analyzing the current state of the formal literature to identify whether grey literature could address relevant limitations (Garousi et al. 2019). To support this decision, they propose a checklist of seven Yes/No questions. A high number of Yes answers indicates that including grey literature is likely to add value and reduce the risk of an incomplete synthesis (Garousi et al. 2019). Following this guidance, we completed the checklist for LLM usage in QSE (Table 1), answering as follows:

- The first two questions are focused on the topic itself rather than the analysis that will be conducted. For the first question, we answered yes because QSE is widely framed as requiring dedicated engineering approaches rather than direct reuse of classical ones, given the characteristics of quantum software and its lifecycle (Murillo et al. 2025). In addition, surveys on the testing and analysis of quantum software emphasize that quantum-specific properties complicate core quality activities and pose distinct challenges for validation and correctness as-

sessment (Paltenghi and Pradel 2024). As for the second question, we answered yes since the two main topics discussed in the study, QSE and LLMs, are relatively young research areas, and their combination further reduces the number of available studies for retrieval and analysis.

- The third question concerns whether contextual information is important for the subject under study. We answered yes because LLM behavior is highly sensitive to how prompts are phrased and formatted, and even minor prompt variations can lead to materially different outputs, including in code generation tasks Sclar et al. (2023). In the QSE domain, this sensitivity is amplified by the heterogeneity of quantum SDKs, which provide different abstractions and development workflows; as a result, what constitutes a correct or idiomatic solution can vary across platforms (Salm et al. 2021; Scekic and Yakaryilmaz 2022).
- Questions four, five, and six are about the work objectives. Our goal is to provide knowledge that is useful both for advancing research on LLM usage in QSE and for informing how suitable is this technology when adopted in practice. In light of this goal, corroborating scientific evidence with practitioner experience is essential, because formal research can offer more systematic and robust insights, while grey and practitioner sources can complement them with concrete, actionable perspectives grounded in real development settings (Garousi et al. 2016, 2019; IBM Quantum 2024). At the same time, academic research is also crucial to critically assess, and when necessary disprove, overly optimistic or unsupported practitioner claims, particularly in a fast-evolving domain where tool capabilities may outpace rigorous evaluation (Garousi et al. 2016, 2019; Murillo et al. 2025).
- The last question concerns practitioners' interest in the matter. We answered yes because LLM usage in QSE sits atop an unusually active practitioner ecosystem that produces a large amount of public, practice-oriented material (open-source code, tutorials, notebooks, announcements). Major quantum SDK communities maintain substantial GitHub ecosystems and learning resources, such as Qiskit's core repository and tutorial collections(Qiskit contributors), Cirq's framework and notebooks(Google Quantum contributors), and PennyLane's libraries and demo notebooks(PennyLaneAI), indicating sustained practitioner engagement and continuous knowledge production beyond peer-reviewed venues. Additionally, tools like Qiskit Code Assistant, which explicitly integrate LLMs into quantum programming workflows, further signal strong practitioner interest in LLM-enabled QSE tasks.

Research Question Definition. To reach our goal, we defined the following research questions that guided the study:

> **RQ$_1$**—Which Quantum Software Engineering tasks are supported by Large Language Models?

This question aims to identify the specific QSE tasks for which LLMs are employed. While RQ_1 focuses on broader research areas, this question investigates the concrete activities performed within those areas. Mapping LLM support at the task level enables us to better understand how these models are operationalized in practice and to identify which development activities receive the most attention and which remain underexplored.

> **RQ₂**—Which Large Language Models are used in Quantum Software Engineering studies?

This question investigates the specific models adopted in the QSE literature. Different LLM families may offer distinct capabilities and trade-offs between performance, reproducibility, and cost. Analyzing which models are used provides insights into current practices and future directions for tool support in QSE.

> **RQ₃**—What limitations emerge from the usage of LLMs in Quantum Software Engineering literature?

This question focuses on the limitations of LLMs in the QSE context reported in the literature. Documenting the limitations of LLMs is useful both for analyzing the current boundaries of what LLM-based approaches can reliably do in QSE and for guiding future improvements in the field.

3.1.2 Search Strategy

The Search strategy comprises two subphases.

Data Sources Selection. This phase aims to identify the most reliable databases to extract the literature and start our process. This step differs between white and grey literature.

- *White Literature.* To collect white literature, we searched for studies indexed in IEEE Xplore, ACM Digital Library, and Scopus. These sources are widely regarded as three of the most prominent computer science databases for white literature[1] and have been adopted in several prior literature reviews (Garousi and Mäntylä 2016b; Garousi et al. 2017; Iuliano and Di Nucci 2024; Lambiase et al. 2024). Furthermore, they are recommended by established guidelines (Garousi et al. 2019; Keele et al. 2007) for their broad coverage of published research. Following (Garousi et al. 2019; Keele et al. 2007), we exported all records returned by the initial search and then applied our eligibility criteria to filter them. We did not impose timeframe constraints, so that all relevant studies published up to the search date could be included.

[1] `https://paperpile.com/g/research-databases-computer-science`

– *Grey Literature.* To retrieve grey literature, we relied on the Google search engine. We made this choice because Google has been used in prior SE MLRs (Garousi and Mäntylä 2016b; Garousi et al. 2017; Islam et al. 2019) and is also recommended by multiple MLR guidelines (Adams et al. 2017; Garousi et al. 2019). At the same time, establishing a stopping rule for GL searches is challenging, largely due to the high volume of available material (Garousi et al. 2019). Therefore, in line with Garousi et al. (2019), we adopted two complementary stopping criteria: (1) we stopped when the search results no longer introduced new concepts (theoretical saturation), and (2) we applied an effort-bounded strategy by considering only the top N search hits. In practice, we screened the first 10 Google search results pages, as the search algorithm typically presents the most relevant items on the initial pages.

Search String Design. In this phase, we defined the search string used to retrieve both white and grey literature. We developed it using insights from the initial examination conducted in Section 3.1.1, which helped us identify the key concepts and the most frequently used terminology in the field. Specifically, the string was constructed by combining (1) key terms describing the topic of interest (extracted from the initial set of articles analyzed), (2) synonyms, spelling variants, and alternative expressions to improve coverage, and (3) logical operators (e.g., AND/OR) and grouping rules to represent different combinations of terms while preserving the intended meaning.
In this way, we developed the following search string:

("quantum*" ∨ "quantistic") ∧
("software engineering" ∨ "software development") ∧
("llm" ∨ "large language models" ∨ "generative ai" ∨
"genai" ∨ "chatgpt" ∨ "llama" ∨ "gemini" ∨ "qwen")

For white literature sources, the search string was applied to the title, abstract, and full text fields across the three selected databases. For grey literature, the search string was queried across all fields in ArXiv, whereas in Google Search, the same string was entered directly in the search bar without modification.

3.1.3 Eligibility Criteria

This phase aims to define criteria to be applied after literature collection to filter the retrieved studies and identify those that provide direct evidence to address our **RQ**s. It is organized into two sub-phases: (1) the definition of eligibility criteria for white literature and (2) the definition of eligibility criteria for grey literature (Garousi and Mäntylä 2016b; Garousi et al. 2019).

For white literature, according to the guidelines by Kitchenham et al. (2007) and prior studies (Iuliano and Di Nucci 2024; Lambiase et al. 2024), we structured the eligibility assessment into three complementary sets of criteria.

Table 2: Inclusion criteria used in the selection process.

ID	Inclusion Criteria
IC-1	The study explicitly addresses Quantum Software Engineering (QSE) tasks.
IC-2	The paper discusses, applies, or evaluates Large Language Models in the context of QSE.

Table 3: Exclusion criteria used in the selection process.

ID	Exclusion Criteria
EC-1	The article is incomplete (poster, proceedings paper, or presentation without full content).
EC-2	The paper is not written in English.
EC-3	The paper is a secondary study.
EC-4	The paper is short (fewer than 4 pages, abstract-only papers, or extended abstract).
EC-5	The focus of the paper is not on using LLMs in QSE.

Table 4: Quality assessment criteria.

ID	Quality Criterion
Q-1	Does the paper clearly describe the QSE task being assessed and the role of LLMs?
Q-2	Does the paper clearly describe the empirical results, benchmark, or evaluations?

The eligibility assessment is structured around three complementary sets of criteria. First, the **Exclusion Criteria** (see Table 3) define the conditions under which a study is removed from the analysis set when at least one criterion is satisfied. Second, the **Inclusion Criteria** (see Table 2) specify the conditions that permit a study to be considered for inclusion once at least one criterion is met. Finally, the **Quality Criteria** (see Table 4) are used to evaluate the overall rigor and relevance of each study. All the criteria have been assessed on a binary scale (Yes = 1, No = 0), and only papers that satisfied both criteria were retained for the final evaluation.

Regarding the evaluation of grey literature, we followed two distinct paths: for grey literature extracted from ArXiv, we used the same eligibility criteria as for white literature, since the sources were preprints and thus those criteria could be readily applied. As for the grey literature extracted from Google Search, we followed the criteria proposed by Garousi et al. (2019) that are summarized in Table 5 for each source. As also defined in the article by Garousi et al. (2019), each of the criteria was evaluated on the sources using a 3-point Likert scale (Yes = 1, Partly = 0.5, and No = 0). Following similar studies (Lambiase et al. 2024), we excluded articles that obtained a combined score among all the criteria ≤ 4.

Table 5: Eligibility Criteria for Grey Literature defined by Garousi et al. (2019)

Criterion	Evaluation Questions
Authority of the Producer	Is the publishing organization reputable? Is the author associated with a reputable organization? Has the author published other work in the field? Does the author demonstrate expertise in the area?
Methodology	Is there a clearly stated aim? Is the methodology described? Are authoritative and contemporary references provided? Are limitations clearly stated? Does the work address a specific question or case?
Objectivity	Is the presentation balanced? Are statements objective rather than opinion-based? Are there any vested interests? Are conclusions supported by data?
Date	Is the publication date clearly stated?
Novelty	Does the work provide a unique contribution? Does it strengthen or refute an existing position?
Related Sources	Are key related grey or formal sources discussed or linked?
Outlet Type	Tier 1 (High credibility): Books, theses, government reports, white papers. Tier 2 (Moderate credibility): Reports, news articles, presentations, videos, Q/A sites, wikis. Tier 3 (Low credibility): Blogs, emails, social media posts.

After applying the eligibility criteria to all identified sources, we further expanded the set of relevant studies through backward and forward snowball sampling. Specifically, backward snowballing was conducted by examining the reference lists of the included studies, while forward snowballing was performed by identifying newer publications that cite the selected papers.

3.2 Conducting Phase

3.2.1 Study Selection Phase

During the study selection phase, the predefined search and screening strategy was applied to systematically collect and refine the body of literature, following established guidelines for conducting secondary studies in software engineering and evidence-based research (Garousi and Mäntylä 2016b; Garousi et al. 2019). The analysis considered all studies available as of December 3, 2025, ensuring comprehensive coverage of the relevant evidence published at that time.

The application of eligibility criteria was independently conducted by the first two authors to mitigate selection bias and enhance the reliability of the screening process. To support collaboration and conflict resolution during

screening, we used the Rayyan web platform (Rayyan), a tool for conducting systematic reviews.

The first author executed the search across the selected digital libraries for both white and grey literature using the search string defined in 3.1.2. For digital library sources, the search string was applied to the title, abstract, and, where supported, full-text fields to ensure comprehensive retrieval. For grey literature identified through Google Search, the first author entered the same search string directly into the search engine and followed the eligibility procedure described in 3.1.3. In line with the guidelines by Garousi et al. (2019), the screening process continued until theoretical saturation was reached.

The search process yielded 172 records across white and grey literature sources. For white literature, the search retrieved 18 studies from IEEE eXplore, 95 from the ACM Digital Library, and 12 from Scopus, for a total of 125 records. As for the grey literature, the search yielded 41 studies from ArXiv and 6 sources from Google Search, totaling 47 grey literature records. After aggregating all retrieved studies from both white and grey literature, we removed duplicate entries across sources, reducing the dataset from 174 to 154 unique studies, 114 from the white and 40 from the grey literature, which constituted the screening set.

The screening was conducted in two sequential phases for white and grey literature by the first two authors.

For the **white literature**, the set of studies was divided into two subsets, and the first two authors independently applied the inclusion and exclusion criteria to their respective portions. For each study, the decision to include or exclude was based on the title and abstract, and, when necessary, extended to a full-text examination. After each author had screened an initial batch of 10 studies, they met to compare their decisions and discuss potential discrepancies. No conflicts emerged, confirming a consistent interpretation of the eligibility criteria. The authors then continued the screening process independently on the remaining studies. After applying inclusion and exclusion criteria, 8 articles were retained, and 105 articles were excluded: 59 articles were excluded by EC-1, 5 by EC-3, 1 by EC-4, and 39 by EC-5. Subsequently, a joint validation session was conducted to reassess the included studies and minimize the risk of erroneous inclusions. Finally, the first two authors applied the predefined quality criteria to the retained papers, and 2 articles failed to meet Q-2 criteria, leaving a final set of 7 white literature studies.

Regarding the **grey literature**, we started by applying the eligibility criteria to the papers retrieved from ArXiv. For this process, the same general procedure was followed, so the first two authors independently applied the inclusion and exclusion criteria through a two-step process of title and abstract screening, followed by full-text assessment when needed. This process resulted in 12 studies included and 21 excluded: 20 were excluded by EC-5, and 1 by EC-4. After applying the quality criteria, 2 studies were excluded, 1 by Q-1 and 1 by Q-2, resulting in a final set of 9 grey literature studies extracted from ArXiv. As for the grey literature results extracted from Google Search, the first author conducted the search on December 13, 2025, yielding 6 results. After

that, the first two authors applied the quality criteria in Table 5, as defined by Garousi et al. (2019), and excluded 2 sources, resulting in 4 valid sources.

After completing the searches of both white and grey literature, we applied backward and forward snowballing across the combined set of studies. Backward snowballing identified 4 additional candidate papers, while forward snowballing yielded 3 further studies. The first two authors applied the defined inclusion, exclusion, and quality criteria to these candidates and identified 6 additional primary studies.

At the end of the process, a final set of 26 sources composed the articles used to address the research questions. Detailed information on all screened and excluded studies is provided in the replication package Replication Package.

3.2.2 Data Extraction Phase

Starting from the research questions and following established MLR guidelines (Garousi et al. 2016, 2019), we developed a structured data extraction form to systematically collect information from each included study. Consistent with prior secondary studies in SE, the extraction form was collaboratively defined through iterative discussions among the authors. It was then piloted on a subset of studies and refined until consensus was reached regarding the extracted fields and their interpretation.

Other than general demographic data, for each RQ we collected different information. The data form is summarized in Table 6.

- For the first research question (RQ_1), concerning QSE areas and tasks, we adopted a deductive approach based on the taxonomy proposed by Weder et al. (2022). Specifically, each study was examined to identify the lifecycle phase(s) addressed and the corresponding tasks falling within the components defined in the taxonomy.
- For the second research question (RQ_2), focusing on the LLMs used in QSE, we extracted the specific models explicitly reported in each study and recorded additional contextual notes when necessary (e.g., version, configuration, or usage setting).
- For the third research question (RQ_3), addressing limitations and challenges, we applied an inductive in-vivo coding approach, systematically identifying and coding all limitations explicitly discussed by the authors of the primary studies.

After stabilization of the extraction protocol, the first author applied the form to all included studies. To enhance reliability and reduce potential bias, a second author independently reviewed a subset of the extracted data. Any discrepancies were discussed until agreement was reached.

3.2.3 Data Synthesis and Analysis

After completing the data extraction phase, we synthesized and analyzed the collected evidence to derive structured answers to each research question. The

Table 6: Data Extraction Form

Dimension	Description
Bibliographic Information	Authors, year of publication, and venue.
Task Performed	Nature of LLM support (e.g., code generation, bug fixing, test generation).
QSE Lifecycle Activity	Quantum Software Engineering phase addressed by the study, classified according to the taxonomy by Weder et al. (2022) (e.g., Implementation, Analysis).
LLM Family and Model	Large Language Model(s) used in the study (e.g., GPT, LLaMA), including the specific model version when reported.
LLM Availability Type	Whether the adopted model is open-source or closed-source.
Reported Limitations	Limitations explicitly discussed in the study regarding the use of LLMs in QSE (e.g., correctness issues, data coverage, evaluation scope, prompt dependency).

synthesis process was conducted through iterative discussions between the first two authors, who systematically reviewed the extracted data and agreed on the most appropriate representation strategy for each RQ.

For $\mathbf{RQ_1}$, given the prior adoption of the taxonomy by Weder et al. (2022), the primary synthesis effort focused on consolidating the identified tasks. When different studies used heterogeneous terminology to describe conceptually similar activities, these were harmonized into unified task categories. The final representation follows a two-level hierarchical structure: the top level corresponds to the lifecycle phases defined by Weder et al. (2022), while the second level captures the specific QSE tasks supported by LLMs.

For $\mathbf{RQ_2}$, the synthesis centered on the LLMs employed in the studies. Models were grouped and reported according to their name, relevant characteristics (e.g., model family or version when available), and frequency of occurrence across the dataset.

For $\mathbf{RQ_3}$, which concerns limitations and challenges, a more interpretive synthesis was required due to the heterogeneity of reported issues. The extracted limitations were iteratively compared and clustered into higher-level themes through constant discussion between the two authors, resulting in a set of consolidated categories representing recurring challenges in the literature.

Finally, descriptive statistics were computed to complement the qualitative synthesis. These statistics were used to highlight observable trends, such as the most frequently addressed lifecycle phases, the predominant LLM families, and the most commonly reported categories of limitations.

Table 7: Distribution of primary studies by QSE lifecycle phase (based on Weder et al. (2022)).

Lifecycle Phase	**# Papers**	**Items**
Implementation	8	(Basit et al. 2025a,b; d'Aloisio et al. 2024; Henderson et al. 2025; Kheiri et al. 2025) (Campbell et al. 2025; Deroy and Maity 2024) (Ezratty 2024; News 2025)
Analysis	8	(d'Aloisio et al. 2024; Easttom 2024; Guo et al. 2024; Dupuis et al. 2025; Suárez et al. 2025a,b) (Aragonés-Soria and Oriol 2025; Shin et al. 2025) (Shah 2023)
Testing	2	(Xia et al. 2024) (Sivaloganathan et al. 2024)
Architecture and Design	1	(Liang et al. 2023)

4 Results

In this section, we present the results for each research question and discuss the evidence from the selected studies, highlighting the main findings and their implications for the QSE field.

4.1 **RQ$_1$**—Which Quantum Software Engineering tasks are supported by Large Language Models?

To address **RQ$_1$**, we adopted a two-level classification strategy. First, we mapped each primary study to the phases of the quantum software development lifecycle proposed by Weder et al. (2022). Second, within each lifecycle phase, we identified the specific QSE tasks supported by LLMs. This approach enables a theory-grounded macro-level analysis while preserving task-level granularity. Overall, the evidence base is unevenly distributed across the lifecycle (Table 7). Although the taxonomy of Weder et al. (2022) comprises seven phases, the selected studies concentrate on four: Implementation, Analysis, Testing, and Architecture/Design. No evidence was found for requirements engineering, deployment, or maintenance beyond code-level evolution, indicating substantial research gaps.

4.1.1 Implementation

Weder et al. (2022) defines implementation as the phase in which "the quantum application is implemented based on the requirements and design from the previous phases", including the development of the constituting software artifacts. We identified 8 studies (5 white literature, 3 grey literature) targeting this phase. The dominant task is **quantum code and circuit generation**, with several studies evaluating LLMs (e.g., GPT-4) for automatically producing quantum programs or parameterized circuits (Basit et al. 2025a,b;

Henderson et al. 2025; Kheiri et al. 2025)(Campbell et al. 2025; Deroy and Maity 2024). Some works extend generation to structured settings, such as QAOA-based optimization, or adopt multi-agent paradigms to iteratively refine synthesized programs.

Beyond generation, a subset of studies explores **LLM-assisted implementation support**, including prompt-driven refinement and interactive assistance during circuit construction (d'Aloisio et al. 2024)(Ezratty 2024; News 2025). These findings indicate that implementation—particularly code synthesis—is currently the primary entry point for LLM adoption in QSE.

4.1.2 Analysis

The analysis phase aims to "find bugs that have to be fixed or possible improvements for the quantum application" Weder et al. (2022). We identified 8 studies (5 white literature, 3 grey literature) in this category.

Within this phase, we identified multiple tasks:

- **Program repair and automated fixing**, including LLM-driven correction of faulty quantum programs (Guo et al. 2024).
- **Refactoring and code evolution**, such as structured migration across Qiskit versions and taxonomy-driven refactoring (Suárez et al. 2025a,b).
- **Code optimization and executability enhancement**, supporting hardware-aware improvements (Easttom 2024; Dupuis et al. 2025).
- **Program comprehension and explanation**, where LLMs provide natural language explanations of circuits or architectures (d'Aloisio et al. 2024)(Aragonés-Soria and Oriol 2025; Shin et al. 2025).
- **Linting and quality assurance support**, adapting traditional static analysis concepts to quantum development (Shin et al. 2025).

Additionally, some grey literature discusses **ethical and legal considerations** of LLM-generated quantum code, including intellectual property risks and ambiguity in generated artifacts (Shah 2023).

Compared to Implementation, the Analysis phase reveals a broader diversity of tasks, suggesting an emerging shift from pure generation toward quality-oriented support activities.

4.1.3 Testing

Testing is defined as verifying the intended behavior of a quantum application before delivery Weder et al. (2022). Only 2 studies (1 white, 1 grey) address this phase.

The identified tasks include:

- **Fuzz testing for quantum programs**, extending classical fuzzing techniques to the quantum domain (Xia et al. 2024).

– **Detection of flaky tests**, addressing reliability issues stemming from probabilistic execution and hardware noise (Sivaloganathan et al. 2024).

Despite the centrality of testing in software engineering, its coverage remains minimal, indicating a significant research opportunity.

4.1.4 Architecture and Design

Only one grey literature study addresses architecture and design. (Liang et al. 2023) introduces QGAS, where GPT-4 iteratively proposes and refines ansatz architectures for Variational Quantum Algorithms based on performance feedback. This suggests that LLMs can guide architectural exploration, but evidence remains preliminary.

Answer to $\mathbf{RQ_1}$

The current evidence base on LLMs in QSE is heavily concentrated in the Implementation and Analysis phases, particularly in quantum code generation and code-level maintenance tasks. Testing and Architecture/Design remain sparsely explored, while several lifecycle phases are entirely unaddressed. This indicates that LLM adoption in QSE is still generation-centric, with limited attention to lifecycle-wide integration.

4.2 $\mathbf{RQ_2}$—Which Large Language Models are used in QSE studies?

To answer $\mathbf{RQ_2}$, we extracted and categorized the Large Language Models adopted in the selected studies. The models were grouped by family (e.g., GPT, LLaMA, StarCoder, DeepSeek) and type (open-source or closed-source) to highlight which LLMs are most frequently used in QSE research and to contrast proprietary and open alternatives.

The analysis of the included studies exposed in Table 8 shows that the GPT family dominates the field, appearing in a majority of papers. GPT-3.5 has been widely used for diverse tasks such as program repair, comprehension, and code generation (d'Aloisio et al. 2024; Easttom 2024)(Guo et al. 2024)(Deroy and Maity 2024; Shin et al. 2025). At the same time, GPT-4 and its more recent versions (GPT-4o, GPT-4o-mini, GPT-4.1) have been applied in studies on testing, taxonomy generation, and automated coding support (Henderson et al. 2025)(Xia et al. 2024)(Sivaloganathan et al. 2024; Suárez et al. 2025a,b)(Ezratty 2024; News 2025). The presence of GPT-OSS as an open-source variant (Dupuis et al. 2025) illustrates ongoing efforts to replicate the capabilities of proprietary GPT models while improving reproducibility.

Other closed-source models, such as Gemini, are less common, as they have appeared only once (Deroy and Maity 2024). This suggests that, outside the GPT family, closed-source models have not yet gained a strong foothold in QSE research.

Table 8: Distribution of LLMs in QSE studies.

LLM Family	**LLM Name**	**LLM Type**	**Items**
GPT	GPT-3.5	Closed-Source	(d'Aloisio et al. 2024; Easttom 2024)(Guo et al. 2024)(Deroy and Maity 2024; Shin et al. 2025)
	GPT-4	Closed-Source	(Henderson et al. 2025)(Xia et al. 2024)(Suárez et al. 2025a)(Ezratty 2024; News 2025)
	GPT-4o-mini	Closed-Source	(Sivaloganathan et al. 2024; Suárez et al. 2025b)
	GPT-4o	Closed-Source	(Sivaloganathan et al. 2024; Suárez et al. 2025b)
	GPT-4.1	Closed-Source	(Guo et al. 2025)
	GPT-OSS	Open-Source	(Dupuis et al. 2025)
Deepseek	Deepseek V3	Open-Source	(Guo et al. 2025; Suárez et al. 2025b)
	Deepseek R1	Open-Source	(Guo et al. 2025; Suárez et al. 2025b)
	Deepseek Coder	Open-Source	(Vishwakarma et al. 2024)
StarCoder	StarCoder 2	Open-Source	(Vishwakarma et al. 2024; Xia et al. 2024)(Kheiri et al. 2025; Campbell et al. 2025)
Claude	Claude 3.7 Sonnet	Closed-Source	(Guo et al. 2025)
LLaMa	Tiny-LLaMa	Open-Source	(d'Aloisio et al. 2024)
	LLaMa 2	Open-Source	(d'Aloisio et al. 2024)
	LLaMa 3.1	Open-Source	(Deroy and Maity 2024)
	LLaMa 3.3	Open-Source	(Basit et al. 2025a)
	LLaMa 4 Maverick	Open-Source	(Guo et al. 2025)
	CodeLLaMa	Open-Source	(Vishwakarma et al. 2024; Guo et al. 2025)
Granite	Granite	Open-Source	(Vishwakarma et al. 2024; Kheiri et al. 2025; Campbell et al. 2025)
BERT	BERT	Open-Source	(Aragonés-Soria and Oriol 2025)
Qwen	Qwen 2.5	Open-Source	(Dupuis et al. 2025; Guo et al. 2025)
	Qwen 3	Open-Source	(Dupuis et al. 2025)
Mistral	Mistral Small 3.2	Open-Source	(Dupuis et al. 2025)
Gemini	Gemini	Closed-Source	(Deroy and Maity 2024; Guo et al. 2025)
	CodeGemma	Open-Source	(Vishwakarma et al. 2024)

Table 9: Limitations identified in the primary studies.

Theme	Items
Data Availability and Coverage	(Xia et al. 2024)(Aragonés-Soria and Oriol 2025; Campbell et al. 2025)
Performance Limitations	(Kheiri et al. 2025; Shin et al. 2025; Sivalo-ganathan et al. 2024)
Correctness and Reliability	(Henderson et al. 2025)(Deroy and Maity 2024) (Ezratty 2024; Insider 2025; News 2025)
Comprehension and Explanation Quality	(d'Aloisio et al. 2024) (Guo et al. 2024)
Prompt Dependency	(Suárez et al. 2025b)
Output Alignment Issues	(Suárez et al. 2025a)
Model Constraints	(Basit et al. 2025a)
Evaluation Scope Limitations	(Easttom 2024)(Dupuis et al. 2025)

In contrast, several open-source families are increasingly represented. LLaMA models, ranging from Tiny-LLaMA to LLaMA 3.3 and LLaMa 4, were used in three studies (d'Aloisio et al. 2024)(Basit et al. 2025a; Deroy and Maity 2024), often in combination with other models. Similarly, DeepSeek models (V3 and R1) were applied in tasks related to taxonomy generation and code maintenance (Suárez et al. 2025b). StarCoder 2 (Xia et al. 2024)(Kheiri et al. 2025; Campbell et al. 2025) and Granite (Kheiri et al. 2025; Campbell et al. 2025) also appear in multiple works, particularly for code generation. Beyond these, a few exploratory studies employed Qwen (Dupuis et al. 2025), Mistral Small (Dupuis et al. 2025), or BERT (Aragonés-Soria and Oriol 2025), though always in a limited scope.

Answer to RQ_2

The GPT family clearly dominates the current QSE literature, with GPT-3.5 and GPT-4 variants being the most frequently adopted models across implementation, analysis, and testing tasks. While several open-source alternatives (e.g., LLaMA, DeepSeek, StarCoder, Granite, Qwen, Mistral) are increasingly explored, their usage remains fragmented and often complementary to GPT-based evaluations. Overall, QSE research is currently characterized by a strong reliance on proprietary models, with emerging but still limited efforts toward open and reproducible LLM-based experimentation.

4.3 ⚖ RQ_3—What limitations are reported in QSE studies that employ LLMs?

To answer RQ_3, we performed a thematic analysis of the limitations reported in the selected studies. Each limitation statement was first examined through

open coding, where short descriptive labels were assigned (e.g., low coverage due to missing training data, hallucinations in generated circuits). Similar codes were then grouped into broader categories, which were iteratively refined into higher-level themes that captured recurring issues across studies. This process allowed the analysis to move systematically from individual pieces of evidence to an aggregated view of the limitations of LLMs in QSE.

The analysis revealed eight themes, reported in Table 9. A recurring issue concerns data availability and coverage, with studies noting that important quantum features are often missing from training data, making generalization difficult (Xia et al. 2024)(Aragonés-Soria and Oriol 2025; Campbell et al. 2025). Related to this are performance limitations, as many models perform well on simple or well-known tasks but degrade significantly on more complex or large-scale problems (Kheiri et al. 2025; Shin et al. 2025; Sivaloganathan et al. 2024).

Another major challenge is correctness and reliability, with several studies reporting hallucinations, semantically incorrect code, or failures in hardware simulators that undermine trust in the generated solutions (Henderson et al. 2025)(Deroy and Maity 2024)(Ezratty 2024; Insider 2025; News 2025). Similarly, comprehension and explanation quality were found to be limited: explanations were often shallow, inconsistent, or required prior context to be useful for developers (d'Aloisio et al. 2024; Guo et al. 2024).

Some limitations are process-related. Papers highlighted the need for prompt dependency, where effective results could only be obtained through carefully engineered instructions (Suárez et al. 2025b). This connects to output alignment issues, where suggestions were either too specific or failed to match the actual developer needs (Suárez et al. 2025a).

Finally, model constraints were reported, particularly the under-performance of smaller models and the fact that some results could only be achieved with certain open-source variants (Basit et al. 2025a). A further methodological theme is evaluation scope limitations, as many studies restricted their validation to well-known algorithms or simplified benchmarks, raising concerns about the generalizability of results to realistic QSE contexts and domains (Easttom 2024)(Dupuis et al. 2025).

Answer to RQ_3

The reported limitations cluster around eight recurring themes, with correctness and reliability, data availability, and performance degradation emerging as the most critical concerns. Several studies highlight hallucinations, semantically incorrect quantum code, and unstable behavior across complex tasks. Additional challenges relate to prompt sensitivity, output alignment, and limited evaluation scopes restricted to simplified benchmarks. Overall, the current evidence suggests that while LLMs show promise in QSE, their application remains fragile, highly context-

dependent, and not yet robust enough for fully autonomous use in realistic quantum development settings.

5 Discussions and Implications

Our results highlight a set of observations worth discussing, as they clarify where LLM support for QSE is currently strongest, where it is absent, and what this means for both research and practice. In the following paragraphs, we report the observations and implications from our review, organized by topic. Following the suggestions in Garousi et al. (2019), an MLR should provide concrete suggestions for both researchers and practitioners; in the rest of this section, we represent the implications for researchers using the symbol 📝, and those for practitioners using the symbol 👉.

5.1 LLMs and lifecycle coverage: why the evidence is clustered near code

By mapping studies to the quantum software development lifecycle taxonomy in Weder et al. (2022), we found that the current evidence base is concentrated in only three phases (Implementation, Analysis, and Testing), leaving the remaining phases underrepresented in our dataset. This imbalance is not merely descriptive; it suggests that the community implicitly treats LLMs as code-centric assistants rather than as lifecycle-level engineering instruments. While this finding is largely consistent with what occurs in classical SE (Sergeyuk et al. 2025), in practice, this limits adoption because the value of automation is realized only when outputs can be carried across phases with traceability and confidence, rather than being evaluated in isolation. A clear example is the emphasis on implementation tasks, where LLMs are primarily used to draft circuits or generate quantum code (Basit et al. 2025a; Henderson et al. 2025; Kheiri et al. 2025), sometimes through more complex multi-agent generation schemes (Campbell et al. 2025). Yet, the most recurrent limitations reported across the dataset relate to correctness, reliability, and executability, especially under realistic constraints (Henderson et al. 2025; Dupuis et al. 2025)(Ezratty 2024; News 2025).

A plausible explanation for this phenomenon lies in the still highly abstract nature of quantum technologies. Current quantum development frameworks remain predominantly low-level, requiring direct manipulation of quantum circuits and domain-specific constructs. This technical barrier likely affects the compatibility and usability of the technology, pushing developers to rely on LLMs primarily as support tools for artifact generation rather than as broader engineering assistants across the lifecycle (Lambiase and De Lucia 2026).

👉 **Practitioners** should progressively experiment with LLM support beyond code generation tasks, integrating these tools into earlier and later lifecycle phases (e.g., analysis and testing). Doing so may provide feedback loops that ultimately improve the quality and robustness of generated quantum artifacts.

👉 **Practitioners** must exercise caution when using LLMs for quantum code generation, given the current limitations related to correctness, reliability, and executability under realistic constraints.

📖 **Researchers** should prioritize the development of more robust tools and evaluation frameworks for LLM-assisted quantum code generation, as this phase currently represents both the primary adoption entry point and the area of greatest technical fragility.

5.2 Model choices and reproducibility: a research design implication, not just a tooling preference

The dominance of GPT-family models in the reviewed studies indicates that QSE evidence is currently shaped by a small set of proprietary systems, with open-weight alternatives used less consistently across tasks. A plausible explanation for this dominance is relatively straightforward. Since LLMs already exhibit weaker performance in quantum software tasks compared to classical software engineering—largely due to the scarcity of quantum-specific training data—it is unsurprising that the community gravitates toward models that are widely regarded as more capable and performant. In an already challenging domain, researchers naturally prefer the most powerful available systems to obtain usable results.

This concentration, however, has methodological implications. Model access constraints and the rapid evolution of proprietary systems can undermine reproducibility and longitudinal comparability, particularly when outcomes are highly sensitive to prompt phrasing and evaluation scope (Suárez et al. 2025b)(Easttom 2024).

A key implication for researchers is the need to adopt stronger reporting and replication practices. When feasible, studies should include at least one open-weight baseline to improve transparency and replicability. Moreover, prompts, decoding parameters, and evaluation settings should be explicitly documented, in line with emerging guidelines for LLM usage in Software Engineering research (Baltes et al. 2025). Finally, results should be stratified by task complexity to mitigate the risk that conclusions are disproportionately influenced by simplified or canonical benchmarks (Dupuis et al. 2025)(Campbell et al. 2025).

Researchers should invest in the development of specialized or fine-tuned models, as well as domain-specific tooling, explicitly tailored to QSE tasks. Reducing reliance on general-purpose proprietary systems may strengthen both performance and methodological rigor.

Researchers should systematically adopt and report established guidelines for LLM usage in empirical studies, given the heightened need for scientific rigor in a domain characterized by technical complexity and limited training data.

We refrain from recommending that practitioners rely primarily on open-weight models at this stage, as the research community has not yet produced sufficiently robust and validated domain-specific alternatives. Strengthening the scientific foundation must precede large-scale industrial adoption.

5.3 Addressing limitations with engineering best practices: what teams can do today

The limitations synthesized in our review point to a consistent pattern: the central challenge is not whether LLMs can produce syntactically valid or "quantum-looking" code, but whether such outputs remain dependable under the constraints that truly matter in QSE—namely semantic correctness, backend executability, and robustness under realistic workloads (Henderson et al. 2025; Dupuis et al. 2025)(Ezratty 2024).

In other words, the fragility reported in RQ_3 reframes LLM adoption from a generation problem to a systems integration problem. The question is no longer "Can the model generate code?" but rather "How should teams integrate model outputs into an engineering workflow that preserves trust and correctness?"

Based on the evidence, we identify three practitioner-oriented practices that can be adopted immediately.

Verification-first integration. When LLMs are used for code generation or repair, outputs should be treated as provisional artifacts that must pass explicit validation gates before integration. These gates may include compilation checks, simulator execution, unit tests, and backend-executability validation. This recommendation directly addresses recurrent reports of hallucinated logic, semantically incorrect circuits, and backend failures (Henderson et al. 2025; Dupuis et al. 2025). LLM outputs should enter the pipeline as candidates—not authoritative solutions.

Small diffs over full solutions. Evidence from refactoring and migration studies suggests that bounded transformations are easier to validate and align more closely with developer intent (Suárez et al. 2025a,b). Instead of requesting full solutions, teams should prefer incremental edits, localized

refactorings, or migration patches. Smaller changes reduce ambiguity, lower cognitive load during review, and mitigate output-alignment issues and prompt sensitivity.

Prompting as configuration. Prompt dependency emerges as a recurring limitation (Suárez et al. 2025b). In practice, prompts should be treated as configurable and versioned artifacts rather than ad-hoc instructions. Teams can define standardized prompt templates tied to specific tasks (e.g., migration, linting, repair) and maintain them alongside build scripts or CI configurations. From a research perspective, prompt sensitivity should be explicitly evaluated as a robustness dimension rather than considered incidental variability.

👉 **Practitioners** should frame the adoption of LLM in QSE should as a controlled engineering integration effort, where verification pipelines, incremental transformations, and prompt standardization are systematically embedded into the development workflow.

Researchers should investigate workflow-level integration patterns, rather than isolated generation accuracy, to design LLM-assisted pipelines that are robust, testable, and backend-aware.

6 Limitations and Threat to Validity

Threats to the validity of a MLR typically stem from inaccuracies in data extraction, incomplete coverage of relevant studies due to search strategy limitations (e.g., search terms or database selection), and potential researcher bias in applying inclusion and exclusion criteria. In this section, we systematically identify and address potential validity threats in line with established guidelines for secondary studies (Garousi et al. 2019; Kitchenham et al. 2010; Wohlin et al. 2012), outlining the mitigation strategies adopted in our work.

Internal Validity Internal validity concerns whether the study design and data selection procedures introduce unintended bias or confounding factors (Wohlin et al. 2012). To mitigate such risks, we followed a structured and previously validated review protocol (Kitchenham et al. 2010), detailed in Section 3. All steps of the process—including search engines, search strings, and inclusion/exclusion criteria—were explicitly defined and reported to ensure transparency and replicability. To reduce the risk of missing relevant studies, our search strategy combined formally defined keywords (including synonyms and alternative terminology used in the field) with backward snowballing on the references of initially selected studies. For formal literature, we relied on comprehensive academic databases (e.g., Scopus). For grey literature, we used

the Google Search Engine, consistent with prior MLRs (Garousi and Mäntylä 2016a; Lambiase et al. 2024). To minimize selection bias, all authors participated in the screening process. Study inclusion was determined through joint discussion and voting, and only studies that clearly met the predefined criteria were finally retained for analysis.

Construct Validity Construct validity refers to the degree to which the operationalization of concepts accurately reflects the underlying theoretical constructs (Wohlin et al. 2012). A potential threat arises from the inclusion of grey literature, which often lacks rigorous empirical validation and may reflect practitioner opinions or experience-based claims. While this may limit the strength of certain findings, the inclusion of grey literature is intrinsic to the rationale of an MLR, which aims to integrate academic and practitioner perspectives. In this sense, incorporating grey literature strengthens the ecological validity of the study and helps compensate for the limited empirical evidence currently available in emerging domains such as LLM-supported QSE.

Conclusion Validity Conclusion validity concerns the credibility and robustness of the inferences drawn from the data (Wohlin et al. 2012). To enhance reliability, the first two authors independently reviewed critical steps of the review process, including study selection and data extraction. Disagreements were resolved through discussion until consensus was achieved among all authors. Furthermore, we adhered to established methodological guidelines for both formal literature reviews (Kitchenham et al. 2010) and grey literature inclusion (Garousi et al. 2019). A structured quality assessment was applied to both white and grey literature sources to ensure consistency and rigor.

External Validity External validity refers to the extent to which the findings can be generalized beyond the analyzed sample (Wohlin et al. 2012). We relied on widely recognized and recommended academic databases to enhance coverage and generalizability (Garousi et al. 2019). However, our review was restricted to primary studies written in English, potentially excluding relevant contributions in other languages. Although this may limit generalizability, English remains the dominant publication language in Software Engineering, and language-based exclusion criteria are common practice in secondary studies.

7 Conclusions

In this multivocal literature review, we synthesized the current evidence on how large language models are being used in Quantum Software Engineering. By mapping the selected studies onto the quantum software development lifecycle taxonomy proposed by Weder et al. (2022), we found that existing research focuses on a limited part of the lifecycle. Across these phases, the literature highlights promising opportunities to reduce developer effort but

consistently reports barriers to reliable adoption, including correctness and executability issues, sensitivity to prompts, limited data coverage, and evaluation designs that often remain limited to simplified tasks. Overall, our findings suggest that LLMs can already provide value as assisted tools within constrained and verifiable workflows, while substantial research is still needed to support the full QSE lifecycle through robust evaluations, stronger quality assurance, and reproducible open baselines.

Declarations

Funding

Not applicable.

Ethical approval

Not applicable.

Informed consent

Not applicable.

Author Contributions

Antonio Della Porta: Conceptualization, Methodology, Data Curation, Investigation, Data Analysis, Writing – Original Draft. **Stefano Lambiase**: Conceptualization, Methodology, Investigation, Data Analysis, Writing – Review & Editing. **Andrea De Lucia**: Supervision, Writing – Review & Editing, Validation. **Fabio Palomba**: Supervision, Writing – Review & Editing, Validation.

Data Availability Statement

The articles extracted and analyzed in this study are made available in an online repository(`doi.org/10.6084/m9.figshare.30042556`).

Conflict of Interest

The authors declare that they have no conflict of interest.

Clinical Trial Number in the manuscript

Not applicable.

References

ACM Digital Library. `https://dl.acm.org/`.

Richard J Adams, Palie Smart, and Anne Sigismund Huff. Shades of grey: guidelines for working with the grey literature in systematic reviews for management and organizational studies. *International journal of management reviews*, 19(4):432–454, 2017.

Shaukat Ali. Quantum software testing 101. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, pages 426–427, 2024.

Shaukat Ali and Tao Yue. Modeling quantum programs: Challenges, initial results, and research directions. In *Proceedings of the 1st ACM SIGSOFT International Workshop on Architectures and Paradigms for Engineering Quantum Software*, pages 14–21, 2020.

Sebastian Baltes, Florian Angermeir, Chetan Arora, Marvin Muñoz Barón, Chunyang Chen, Lukas Böhme, Fabio Calefato, Neil Ernst, Davide Falessi, Brian Fitzgerald, et al. Guidelines for empirical studies in software engineering involving large language models. *arXiv preprint arXiv:2508.15503*, 2025.

Vita Santa Barletta, Danilo Caivano, Anibrata Pal, Michele Scalera, and Manuel A Serrano Martin. Enabling quantum privacy and security by design: Imperatives for contemporary state-of-the-art in quantum software engineering. *Journal of Software: Evolution and Process*, 37(2):e70005, 2025.

Abdul Basit, Nouhaila Innan, Muhammad Haider Asif, Minghao Shao, Muhammad Kashif, Alberto Marchisio, and Muhammad Shafique. Pennylang: Pioneering llm-based quantum code generation with a novel pennylane-centric dataset. *arXiv preprint arXiv:2503.02497*, 2025.

Manuel De Stefano, Fabiano Pecorelli, Dario Di Nucci, Fabio Palomba, and Andrea De Lucia. Software engineering for quantum programming: How far are we? *Journal of Systems and Software*, 190:111326, 2022.

Manuel De Stefano, Fabiano Pecorelli, Dario Di Nucci, Fabio Palomba, and Andrea De Lucia. The quantum frontier of software engineering: A systematic mapping study. *Information and Software Technology*, page 107525, 2024a.

Manuel De Stefano, Fabiano Pecorelli, Fabio Palomba, Davide Taibi, Dario Di Nucci, and Andrea De Lucia. Quantum software engineering issues and challenges: Insights from practitioners. In *Quantum Software: Aspects of Theory and System Design*, pages 337–355. Springer, 2024b.

Nicolas Dupuis, Luca Buratti, Sanjay Vishwakarma, Aitana Viudes Forrat, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. Qiskit

code assistant: Training llms for generating quantum computing code. In *2024 IEEE LLM Aided Design Workshop (LAD)*, pages 1–4. IEEE, 2024.

Kanishk Dwivedi, Majid Haghparast, and Tommi Mikkonen. Quantum software engineering and quantum software development lifecycle: a survey. *Cluster Computing*, 27(6):7127–7145, 2024.

Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pages 31–53. IEEE, 2023.

Antonio García de la Barrera, Ignacio García-Rodríguez de Guzmán, Macario Polo, and Mario Piattini. Quantum software testing: State of the art. *Journal of Software: Evolution and Process*, 35(4):e2419, 2023.

Vahid Garousi and Mika V Mäntylä. When and what to automate in software testing? a multi-vocal literature review. *Information and Software Technology*, 76:92–117, 2016a.

Vahid Garousi and Mika V Mäntylä. When and what to automate in software testing? a multi-vocal literature review. *Information and Software Technology*, 76:92–117, 2016b.

Vahid Garousi, Michael Felderer, and Mika V Mäntylä. The need for multivocal literature reviews in software engineering: complementing systematic literature reviews with grey literature. In *Proceedings of the 20th international conference on evaluation and assessment in software engineering*, pages 1–6, 2016.

Vahid Garousi, Michael Felderer, and Tuna Hacaloğlu. Software test maturity assessment and test process improvement: A multivocal literature review. *Information and Software Technology*, 85:16–42, 2017.

Vahid Garousi, Michael Felderer, and Mika V Mäntylä. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and software technology*, 106:101–121, 2019.

Felix Gemeinhardt, Antonio Garmendia, and Manuel Wimmer. Towards model-driven quantum software engineering. In *2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE)*, pages 13–15. IEEE, 2021.

Google Deepmind. `https://gemini.google.com/`.

Google Quantum contributors. An open source framework for programming quantum computers. `https://quantumai.google/cirq`.

Tony Hoare and Robin Milner. Grand challenges for computing research. *The Computer Journal*, 48(1):49–52, 2005.

Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79, 2024.

IBM Quantum. Introducing qiskit code assistant. Web page, 2024. URL `https://www.ibm.com/quantum/blog/qiskit-code-assistant`. Accessed: 2025-12-30.

IBM Quantum. Qiskit code assistant documentation, 2025. URL https://quantum.cloud.ibm.com/docs/guides/qiskit-code-assistant. Accessed 2025-12-16.

IEEE Xplore. https://ieeexplore.ieee.org/Xplore/home.jsp.

Chadni Islam, Muhammad Ali Babar, and Surya Nepal. A multi-vocal review of security orchestration. *ACM Computing Surveys (CSUR)*, 52(2):1–45, 2019.

Gerardo Iuliano and Dario Di Nucci. Smart contract vulnerabilities, tools, and benchmarks: An updated systematic literature review. *arXiv preprint arXiv:2412.01719*, 2024.

Staffs Keele et al. Guidelines for performing systematic literature reviews in software engineering. Technical report, Technical report, ver. 2.3 ebse technical report. ebse, 2007.

Barbara Kitchenham, Stuart Charters, et al. Guidelines for performing systematic literature reviews in software engineering. *Technical report, EBSE Technical Report EBSE-2007-01*, 2007.

Barbara Kitchenham, Rialette Pretorius, David Budgen, O Pearl Brereton, Mark Turner, Mahmood Niazi, and Stephen Linkman. Systematic literature reviews in software engineering–a tertiary study. *Information and software technology*, 52(8):792–805, 2010.

Will Knight. Serious quantum computers are finally here. what are we going to do with them. *MIT Technology Review. Retrieved on October*, 30:2018, 2018.

Stefano Lambiase and Andrea De Lucia. Quantifying adoption: A sem study of quantum software technology in software development. *Empirical Software Engineering*, 31(3):69, 2026.

Stefano Lambiase, Gemma Catolino, Fabio Palomba, and Filomena Ferrucci. Motivations, challenges, best practices, and benefits for bots and conversational agents in software engineering: A multivocal literature review. *ACM Computing Surveys*, 57(4):1–37, 2024.

Neilson Carlos Leite Ramalho, Higor Amario de Souza, and Marcos Lordello Chaim. Testing and debugging quantum programs: The road to 2030. *ACM Transactions on Software Engineering and Methodology*, 34(5): 1–46, 2025.

Heng Li, Foutse Khomh, Moses Openja, et al. Understanding quantum software engineering challenges an empirical study on stack exchange forums and github issues. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 343–354. IEEE, 2021.

Peixun Long and Jianjun Zhao. Testing multi-subroutine quantum programs: From unit testing to integration testing. *ACM Transactions on Software Engineering and Methodology*, 2024.

Xuan Mao, Zijie Huang, Jianxin Ge, Chao Wang, Wuxu Wang, and Lizhi Cai. Towards defect prediction for quantum software. In *2025 IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, pages 1–8. IEEE, 2025.

Meta Labs. https://www.llama.com/.

Microsoft. https://learn.microsoft.com/it-it/azure/quantum/qsharp-overview.

Enrique Moguel, Javier Berrocal, José García-Alonso, and Juan Manuel Murillo. A roadmap for quantum software engineering: Applying the lessons learned from the classics. In *Q-SET@ QCE*, pages 5–13, 2020.

Juan Manuel Murillo, Jose Garcia-Alonso, Enrique Moguel, Johanna Barzen, Frank Leymann, Shaukat Ali, Tao Yue, Paolo Arcaini, Ricardo Pérez-Castillo, Ignacio García-Rodríguez de Guzmán, et al. Quantum software engineering: Roadmap and challenges ahead. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–48, 2025.

Md Nadim, Mohammad Hassan, Ashis Kumar Mandal, and Chanchal K Roy. Quantum vs. classical machine learning algorithms for software defect prediction: Challenges and opportunities. In *2025 IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, pages 35–42. IEEE, 2025.

OpenAI. https://chatgpt.com/.

Matteo Paltenghi and Michael Pradel. A survey on testing and analysis of quantum software. *arXiv preprint arXiv:2410.00650*, 2024.

PennyLaneAI. Pennylane (github repository). GitHub. URL https://github.com/PennyLaneAI/pennylane. Accessed: 2025-12-30.

Mario Piattini, Guido Peterssen, Ricardo Pérez-Castillo, Jose Luis Hevia, Manuel A Serrano, Guillermo Hernández, Ignacio García Rodríguez De Guzmán, Claudio Andrés Paradela, Macario Polo, Ezequiel Murina, et al. The talavera manifesto for quantum software engineering and programming. In *QANSWER*, pages 1–5, 2020.

Mario Piattini, Manuel Serrano, Ricardo Perez-Castillo, Guido Petersen, and Jose Luis Hevia. Toward a quantum software engineering. *IT Professional*, 23(1):62–66, 2021.

Qiskit contributors. Qiskit: An open-source framework for quantum computing. https://qiskit.org/.

Rayyan. https://new.rayyan.ai/.

Replication Package. https://figshare.com/s/25500835fcedc3bb39a7.

Javier Romero-Álvarez, Jaime Alvarado-Valiente, Enrique Moguel, and Jose Garcia-Alonso. Quantum web services: Development and deployment. In *International Conference on Web Engineering*, pages 421–423. Springer, 2023.

Marie Salm, Johanna Barzen, Frank Leymann, Benjamin Weder, and Karoline Wild. Automating the comparison of quantum compilers for quantum circuits. In *Symposium and Summer School on Service-Oriented Computing*, pages 64–80. Springer, 2021.

Aritra Sarkar. Automated quantum software engineering. *Automated Software Engineering*, 31(1):36, 2024.

Marija Scekic and Abuzer Yakaryilmaz. Comparing quantum software development kits for introductory level education. *Baltic Journal of Modern Computing*, 10(1):87–104, 2022.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How

i learned to start worrying about prompt formatting. *arXiv preprint arXiv:2310.11324*, 2023.

Scopus. `https://scopus.com`.

Agnia Sergeyuk, Yaroslav Golubev, Timofey Bryksin, and Iftekhar Ahmed. Using ai-based coding assistants in practice: State of affairs, perceptions, and ways forward. *Information and Software Technology*, 178:107610, 2025.

Ruslan Shaydulin, Caleb Thomas, and Paige Rodeghero. Making quantum computing open: Lessons from open source projects. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*, pages 451–455, 2020.

Unitary Fund. Quantum open source software survey. `https://unitaryfund.github.io/survey-website/`, 2024. Accessed: 2024-10-14.

David Valencia, Jose Garcia-Alonso, Javier Rojo, Enrique Moguel, Javier Berrocal, and Juan Manuel Murillo. Hybrid classical-quantum software services systems: Exploration of the rough edges. In *International Conference on the Quality of Information and Communications Technology*, pages 225–238. Springer, 2021.

Sanjay Vishwakarma, Francis Harkins, Siddharth Golecha, Vishal Sharathchandra Bajpe, Nicolas Dupuis, Luca Buratti, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. Qiskit humaneval: An evaluation benchmark for quantum code generative models. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 1169–1176. IEEE, 2024.

Benjamin Weder, Johanna Barzen, Frank Leymann, and Daniel Vietz. Quantum software development lifecycle. In *Quantum Software Engineering*, pages 61–83. Springer, 2022.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in software engineering*. Springer Science & Business Media, 2012.

Jiaming Ye, Shangzhou Xia, Fuyuan Zhang, Paolo Arcaini, Lei Ma, Jianjun Zhao, and Fuyuki Ishikawa. Quratest: Integrating quantum specific features in quantum program testing. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1149–1161. IEEE, 2023.

Haibo Yu and Jianjun Zhao. The quantum program dependence graph and its uses in quantum software development. In *2025 IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, pages 48–55. IEEE, 2025.

Jake Zappin, Trevor Stalnaker, Oscar Chaparro, and Denys Poshyvanyk. When quantum meets classical: Characterizing hybrid quantum-classical issues discussed in developer forums. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 674–674. IEEE Computer Society, 2025.

Jianjun Zhao. Quantum software engineering: Landscapes and horizons. *arXiv preprint arXiv:2007.07047*, 2020.

Xudong Zhao, Xiaolong Xu, Lianyong Qi, Xiaoyu Xia, Muhammad Bilal, Wenwen Gong, and Huaizhen Kou. Unraveling quantum computing system architectures: an extensive survey of cutting-edge paradigms. *Information and Software Technology*, 167:107380, 2024.

MLR—White Literature

[WL71] Abdul Basit, Minghao Shao, Muhammad Haider Asif, Nouhaila Innan, Muhammad Kashif, Alberto Marchisio, and Muhammad Shafique. Pennycoder: Efficient domain-specific llms for pennylane-based quantum code generation. In *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 2, pages 229–234. IEEE, 2025a.

[WL72] Abdul Basit, Minghao Shao, Muhammad Haider Asif, Nouhaila Innan, Muhammad Kashif, Alberto Marchisio, and Muhammad Shafique. Qhackbench: Benchmarking large language models for quantum code generation using pennylane hackathon challenges. In *2025 IEEE International Conference on Quantum Artificial Intelligence (QAI)*, pages 316–322. IEEE, 2025b.

[WL73] Giordano d'Aloisio, Sophie Fortz, Carol Hanna, Daniel Fortunato, Avner Bensoussan, Eñaut Mendiluze Usandizaga, and Federica Sarro. Exploring llm-driven explanations for quantum algorithms. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, pages 475–481, 2024.

[WL74] Chuck Easttom. Utilizing chatgpt to improve quantum algorithms. In *2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 0744–0749. IEEE, 2024.

[WL75] Xiaoyu Guo, Jianjun Zhao, and Pengzhan Zhao. On repairing quantum programs using chatgpt. in 2024 ieee/acm 5th international workshop on quantum software engineering (q-se), 2024.

[WL76] Elena R Henderson, Jessie M Henderson, Joshua Ange, and Mitchell A Thornton. Programming quantum computers with large language models. In *Quantum Information Science, Sensing, and Computation XVII*, volume 13451, pages 20–34. SPIE, 2025.

[WL77] Kiana Kheiri, Aamna Aamir, Andriy Miranskyy, and Chen Ding. Qspark: Towards reliable qiskit code generation. 2025.

[WL63] Sanjay Vishwakarma, Francis Harkins, Siddharth Golecha, Vishal Sharathchandra Bajpe, Nicolas Dupuis, Luca Buratti, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. Qiskit humaneval: An evaluation benchmark for quantum code generative models. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 1169–1176. IEEE, 2024.

[WL79] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13, 2024.

MLR—Grey Literature arXiv

[GLA80] Yaiza Aragonés-Soria and Manuel Oriol. Architecture for a trustworthy quantum chatbot. *arXiv preprint arXiv:2503.04875*, 2025.

[GLA81] Charlie Campbell, Hao Mark Chen, Wayne Luk, and Hongxiang Fan. Enhancing llm-based quantum code generation with multi-agent optimization and quantum error correction. *arXiv preprint arXiv:2504.14557*, 2025.

[GLA82] Aniket Deroy and Subhankar Maity. Code generation and algorithmic problem solving using llama 3.1 405b. *arXiv preprint arXiv:2409.19027*, 2024.

[GLA83] Nicolas Dupuis, Adarsh Tiwari, Youssef Mroueh, David Kremer, Ismael Faro, and Juan Cruz-Benito. Quantum verifiable rewards for post-training qiskit code assistant. *arXiv preprint arXiv:2508.20907*, 2025.

[GLA84] Xiaoyu Guo, Minggu Wang, and Jianjun Zhao. Quanbench: Benchmarking quantum code generation with large language models. *arXiv preprint arXiv:2510.16779*, 2025.

[GLA85] Zhiding Liang, Jinglei Cheng, Rui Yang, Hang Ren, Zhixin Song, Di Wu, Xuehai Qian, Tongyang Li, and Yiyu Shi. Unleashing the potential of llms for quantum computing: A study in quantum architecture design. *arXiv preprint arXiv:2307.08191*, 2023.

[GLA86] Seung Yeob Shin, Fabrizio Pastore, and Domenico Bianculli. Quantum program linting with llms: Emerging results from a comparative study. *arXiv preprint arXiv:2504.05204*, 2025.

[GLA87] Janakan Sivaloganathan, Ainaz Jamshidi, Andriy Miranskyy, and Lei Zhang. Automating quantum software maintenance: Flakiness detection and root cause analysis. *arXiv preprint arXiv:2410.23578*, 2024.

[GLA88] José Manuel Suárez, Luis Mariano Bibbó, Joaquin Bogado, and Alejandro Fernandez. Automatic qiskit code refactoring using large language models. *arXiv preprint arXiv:2506.14535*, 2025a.

[GLA89] José Manuel Suárez, Luís Mariano Bibbó, Joaquín Bogado, and Alejandro Fernandez. Taxonomy of migration scenarios for qiskit refactoring using llms. *arXiv preprint arXiv:2506.07135*, 2025b.

MLR—Grey Literature Google

[GLG90] Olivier Ezratty. How ai, llms and quantum science can empower each other? `https://www.oezratty.net/Files/Publications/How%20LLMs%20and%20quantum%20science%20can%20empower%20each%20other.pdf`, 2024.

[GLG91] Quantum Insider. Quantum computers aren't quite ready for llm vibe coders, study finds. `https://thequantuminsider.com/2025/07/02/quantum-computers-arent-quite-ready-for-llm-vibe-coders-study-finds/`, 2025.

[GLG92] Quantum News. Can ai write quantum programs, and how well does it perform? `https://quantumzeitgeist.com/`

can-ai-write-quantum-programs-and-how-well-does-it-perform/, 2025.

[GLG93] Jyoti Kunal Shah. Ethical considerations of llm-driven quantum code generation for optimization tasks. https://www.theamericanjournals.com/index.php/tajet/article/view/6316/5837, 2023.